LANGUAGE FUNDAMENTALS

1. What does overriding a method mean in Java?

Method Overriding happens when a child class provides a specific implementation for a method that is already defined in its parent class. The method name, return type, and parameters must match exactly. This is used to change the behavior of the parent method to suit the child class's needs.

2. Can a class override a static method? Why or why not?

No, you cannot override a **static method**. Static methods belong to the class itself, not to object instances. If you define a static method with the same name in a subclass, it is called **Method Hiding**, and the compiler decides which one to call based on the reference type, not the runtime object.

3. Why can’t we override a private method?

Private methods are not visible to subclasses; they are strictly hidden inside the parent class. Since the child class cannot see the method, it cannot override it. If you define a method with the same name in the child class, Java treats it as a completely new, unrelated method.

4. What rules do we need to follow while overriding a method?

The method signature (name and parameters) must match the parent exactly. The access level cannot be more restrictive (e.g., you cannot change public to private). Finally, the overriding method cannot throw new or broader **checked exceptions** than the original method declared.

5. What is method overloading?

Method Overloading is the ability to define multiple methods with the same name in the same class, provided they have different parameter lists. It allows you to perform similar operations (like print()) on different types of data without creating unique names for every method.

6. How does Java choose which overloaded method should run?

Java uses **static binding** (compile-time resolution) to pick the correct method. The compiler looks at the method name and the specific argument types you passed. It then matches that combination to the corresponding method definition in the class.

7. Can static methods be overloaded?

Yes, **static methods** can be overloaded just like instance methods. You can define multiple static methods with the same name as long as their input parameters are different. The compiler distinguishes them based on the arguments passed.

8. What is a covariant return type?

A **covariant return type** allows an overriding method to return a subclass of the type returned by the original parent method. For instance, if the parent returns Number, the child's override can return Integer. This removes the need for type casting when using the specific child class.

9. What happens if a class uses super() even when it has no parent?

In Java, every class automatically extends the **Object** class if no other parent is specified. Therefore, if you write super() in a class that seems to have no parent, it actually calls the constructor of the Object class. A class never truly has "no parent" (except Object itself).

10. Can we use this or super inside static code?

No, you cannot use this or super inside **static methods** or static blocks. These keywords refer to the *current object instance*, but static code belongs to the class and exists independently of any specific object. Attempting to use them will cause a compilation error.

11. What makes this() different from super() inside constructors?

this() is used to call another constructor within the **same class** (to reuse code). super() is used to call a constructor from the **parent class** (to initialize inherited fields). You can use one or the other as the very first line of a constructor, but you cannot use both together.

12. What is the purpose of the this keyword?

The this keyword serves as a reference to the **current object** executing the code. It is most commonly used to resolve naming conflicts (shadowing) when a method parameter has the same name as a class variable (e.g., this.name = name). It can also be used to pass the current object to other methods.

13. Can we change what this refers to?

No, you cannot assign a new value to this. It is effectively a **final** reference that strictly points to the instance currently running the method. You cannot write a statement like this = new Object();.

14. Why do we use the final keyword in variables, methods, or classes?

We use final to enforce **immutability** or restrict modification. A final variable is a constant and cannot change value; a final method cannot be overridden by subclasses; and a final class cannot be extended (subclassed) at all.

15. Does an abstract class allow constructors?

Yes, an **abstract class** can have constructors. Even though you cannot create an instance of the abstract class directly, the constructor is called when a subclass is instantiated. This is crucial for initializing fields defined in the abstract parent.

16. Is it possible for an abstract class to have static methods?

Yes, an abstract class can contain **static methods**. Since static methods are called using the class name and not an object instance, you can execute them directly (e.g., AbstractClass.staticMethod()) without needing to instantiate the class.

17. Can a class use more than one interface?

Yes, a class can implement **multiple interfaces**. This is Java's way of supporting multiple inheritance of behavior. You simply list the interfaces separated by commas in the class declaration (e.g., implements InterfaceA, InterfaceB).

18. Can an interface have variables and what kind of variables are they?

Yes, an interface can have variables, but they are implicitly **public, static, and final**. They act as global constants rather than instance variables. You cannot declare private or protected variables, nor can you have variables that hold object state in an interface.

19. What is constructor chaining and why is it useful?

Constructor chaining is the practice of calling one constructor from another (using this() or super()). It is useful for avoiding **code duplication**. Instead of writing the same initialization logic in every constructor, you pass arguments to a single "master" constructor that handles the setup.

20. Can an interface extend another interface?

Yes, an interface can extend another interface using the extends keyword. In fact, an interface can extend **multiple** other interfaces at the same time, allowing for a rich hierarchy of behavior definitions.

EQUALS, HASHCODE & WRAPPERS

1. What does it mean when two different objects have the same hashCode?

This scenario is called a **hash collision**. It means that two distinct objects calculate to the same integer value and will be stored in the same "bucket" in a HashMap. It does *not* mean the objects are equal; the map must still use .equals() to distinguish between them.

2. What is the rule that connects equals() and hashCode()?

The contract states: if two objects are equal according to .equals(), they **must** have the same hash code. However, the reverse is not required; two objects with the same hash code are not necessarily equal. Violating this rule breaks hash-based collections like HashSet and HashMap.

3. Why should we avoid using mutable objects as keys in HashMap?

If you modify a mutable object (change its fields) after using it as a key, its **hash code** often changes. When you try to retrieve the value later, the Map looks in the wrong "bucket" based

on the new hash code and cannot find your data. This effectively causes a memory leak where data exists but is irretrievable.

4. How does HashMap use equals() during key comparison?

HashMap uses .equals() to resolve **collisions**. First, it uses the hash code to locate the correct memory bucket. If multiple keys reside in that bucket, the Map iterates through them and uses .equals() to check the actual content, ensuring it retrieves the exact key you requested.

5. What happens if we override equals() but forget to override hashCode()?

If you skip hashCode(), two logically equal objects will likely generate different hash codes based on their memory addresses. Consequently, a HashMap will treat them as completely unrelated objects. You won't be able to find a key you inserted using a new, identical object instance.

6. Why is String often used as a key in HashMap?

Strings are **immutable**, meaning their content cannot change, so their hash code remains stable forever. Additionally, the String class calculates its hash code once and caches it. This makes looking up values in a Map extremely fast because the hash doesn't need to be recalculated every time.

7. What issue happens if hashCode() is overridden but equals() is not?

If you don't override .equals(), the default implementation checks memory addresses rather than content. Even if two objects have the same hash code, the Map will see them as different instances. This allows you to accidentally add duplicate objects to a Set or Map, defeating the purpose of unique keys.

8. Can autoboxing cause a NullPointerException?

Yes, specifically during **unboxing** (converting a wrapper object to a primitive). If the wrapper object (like Integer) is null, and you try to assign it to a primitive (int), Java attempts to call .intValue() automatically. Since the object is null, this triggers a NullPointerException.

9. Give an example where autoboxing behaves in a surprising way.

Java caches Integer objects for values between **-128 and 127**. If you compare two Integers with value 100 using ==, it returns true (same cached object). However, if you compare two Integers with value 200 using ==, it returns false (different objects), which confuses developers who expect mathematical equality.

10. How do you correctly write equals() for a simple class like Employee?

First, check if the objects are the same instance (this == obj). Second, check if the other object is null or of a different class. Finally, cast the object to Employee and compare the critical fields (like id and name) to decide if they represent the same data.

COLLECTIONS & DATA STRUCTURES (INTERMEDIATE)

1. How are HashSet and TreeSet different from each other?

HashSet uses a hash table to store elements; it offers the fastest performance (constant time) but does not guarantee any specific order. **TreeSet** uses a Red-Black tree structure; it is slower than HashSet but guarantees that all elements are sorted in ascending order.

2. What makes HashSet different from LinkedHashSet?

HashSet does not maintain any order of elements; they appear randomly. **LinkedHashSet** is very similar but maintains a doubly-linked list across all entries to track the **insertion order**. This means when you iterate through a LinkedHashSet, elements appear exactly in the order you added them.

3. What is ListIterator and how is it different from Iterator?

An **Iterator** can only traverse a collection in the forward direction and can be used with any Collection type (List, Set, etc.). A **ListIterator** extends Iterator but can move both **forward and backward**, and it allows you to modify or add elements during traversal. However, ListIterator works exclusively with classes that implement the **List** interface.

4. What is an IdentityHashMap and when do we use it?

IdentityHashMap is a special map that compares keys using reference equality (==) instead of object equality (.equals()). This means two different objects with the same content are treated as two distinct keys. It is rarely used, typically for topology preservation or serialization frameworks where exact object instances matter.

5. How is Hashtable different from HashMap?

Hashtable is a legacy class that is synchronized (thread-safe), making it slower, and it does not allow any null keys or values. **HashMap** is the modern standard; it is not synchronized (faster) and allows one null key and multiple null values.

6. When should we use LinkedHashMap?

You should use **LinkedHashMap** when you need the speed of a HashMap but also need to maintain the order of entries (either insertion order or access order). It is commonly used to build **LRU (Least Recently Used) caches** because it can automatically move accessed items to the end of the list.

7. What is a WeakHashMap mainly used for?

WeakHashMap is used principally for building caches where you want entries to be automatically deleted if they are not used elsewhere. The keys are stored as "weak references," meaning if the key is no longer referenced by any other part of your program, the Garbage Collector is free to remove the entry from the map.

8. How does fail-fast behavior differ from fail-safe behavior?

Fail-fast iterators (like in ArrayList) throw a ConcurrentModificationException immediately if they detect the collection has changed during iteration. **Fail-safe** iterators (like in CopyOnWriteArrayList) work on a snapshot or copy of the data, allowing the loop to finish without throwing an exception even if data changes.

9. Why does ConcurrentModificationException occur?

This exception occurs when you try to modify a collection (add or remove elements) directly while you are iterating over it using a fail-fast iterator. The iterator keeps a counter of changes; if the collection's structure changes without the iterator's knowledge, it flags this error to prevent unpredictable behavior.

10. How can we avoid ConcurrentModificationException?

To avoid this, you should use the iterator's own .remove() method instead of the collection's remove method. Alternatively, you can use concurrent collections like CopyOnWriteArrayList or ConcurrentHashMap, which handle modifications safely during iteration.

11. How can we sort a list of objects in Java?

You can sort a list using the static method Collections.sort(list) or the list's own .sort() method. The objects inside the list must strictly implement the Comparable interface, or you must pass a custom Comparator to define the sorting logic.

12. Can we sort custom objects without using a Comparator?

Yes, but only if the custom object implements the **Comparable** interface. This interface forces you to override the compareTo() method inside your class, defining the "natural" default sorting order (e.g., sorting Employees by ID).

13. How does Collections.sort() work internally?

Internally, Collections.sort() delegates to the Arrays.sort() method. Since Java 7, it uses an algorithm called **TimSort**, which is a hybrid of Merge Sort and Insertion Sort. It is highly efficient and "stable," meaning it preserves the relative order of equal elements.

14. How can you store items in a Set while keeping the order in which they were inserted?

You should use a **LinkedHashSet**. It implements the Set interface (ensuring uniqueness) but uses a linked list internally to remember the sequence in which elements were added.

15. How can you store elements in sorted form using collections?

To store unique elements in sorted order, use a **TreeSet**. To store key-value pairs sorted by the key, use a **TreeMap**. Both use a Red-Black tree structure to automatically sort data as it is inserted.

16. What is CopyOnWriteArrayList used for?

CopyOnWriteArrayList is a thread-safe variant of ArrayList designed for scenarios where reads are very frequent, but writes are rare. Every time you add or modify an element, it creates a fresh copy of the underlying array. This allows multiple threads to read without locking, preventing interference from writers.

17. What is a BlockingQueue and where is it useful?

A **BlockingQueue** is a thread-safe queue that waits (blocks) if you try to retrieve an item from an empty queue or add an item to a full queue. It is extremely useful for solving **Producer-Consumer** problems, where one thread generates work and another processes it.

18. How do ArrayList, LinkedList, and HashSet differ from each other?

ArrayList is a dynamic array best for fast random access (reading). **LinkedList** is a chain of nodes best for fast insertions and deletions. **HashSet** is a group of unique items best for checking if an item exists (membership) efficiently.

19. What is the difference between Comparable and Comparator?

Comparable is implemented *inside* the class (using compareTo) to define its "natural" default ordering. **Comparator** is a separate, external class (using compare) that allows you to define multiple different sorting strategies (e.g., sort by name, then sort by age) without changing the original object code.

20. What is the difference between Iterator and Enumeration?

Iterator is the modern standard; it works with all collections and allows you to remove elements safely during traversal. **Enumeration** is a legacy interface (used with Vector and Hashtable); it is read-only (cannot remove elements) and has longer method names.

EXCEPTION HANDLING (INTERMEDIATE)

1. What is the difference between throw and throws?

The throw keyword is used inside a method to explicitly trigger an exception instance (e.g., throw new Exception()). The throws keyword is used in the method signature to declare that the method might cause an exception, warning the caller to handle it. throw is an action; throws is a declaration.

2. Can finally run even if we return from try or catch?

Yes, the finally block runs even if there is a return statement in the try or catch blocks. The JVM executes the finally code just before actually returning control to the method caller. It ensures cleanup happens no matter how the method exits.

3. When will the finally block NOT run?

The only common scenario where finally does not run is if you call System.exit() inside the try or catch block, which shuts down the JVM immediately. It also won't run if the JVM crashes (like an infinite loop or memory failure) or if the thread is killed before reaching the block.

4. Can we write try–finally without catch?

Yes, a try-finally block is valid syntax. It is commonly used when you need to guarantee resource cleanup (like closing a lock or file) but do not want to handle the specific error in that method. The exception will propagate up to the caller after the finally block runs.

5. What happens if finally changes the return value?

If the finally block contains a return statement, it overrides any return value from the try or catch blocks. The caller will receive the value returned from finally, and the previous return value is discarded. This is generally considered bad practice because it hides the original result.

6. How can we catch multiple exception types in one catch block?

Starting with Java 7, you can use the pipe symbol (|) to separate multiple exception types within a single catch statement. For example: catch (IOException | SQLException e). This helps reduce code duplication when the handling logic is identical for different errors.

7. Why might finalize() never get called?

The finalize() method is called by the Garbage Collector, but the JVM does not guarantee when or if garbage collection will occur. If your program finishes execution quickly or the system has plenty of memory, the GC might never run, and finalize() will never be executed.

8. How is finally different from finalize()?

finally is a code block used for executing cleanup code after a try-catch structure, regardless of errors. finalize() is a method (now deprecated) that the Garbage Collector calls just before an object is destroyed in memory. One is for flow control; the other is for object lifecycle management.

9. What is exception propagation?

Exception propagation happens when an exception is thrown but not caught in the current method. The JVM drops down the "call stack" to the method that called the current one, looking for a handler. This process repeats until the exception is caught or it reaches the main method and crashes the program.

10. When is a checked exception better than an unchecked one?

Use a checked exception when the error is foreseeable and the calling code can reasonably recover from it (e.g., FileNotFoundException prompts the user to pick a new file). Use an unchecked exception for logical programming errors (like NullPointerException) where the application cannot fix the issue and should likely crash/fail.

11. Can errors and exceptions be handled the same way?

Technically yes, but practically no. Exceptions are application faults that you should catch and handle. Errors (like OutOfMemoryError) indicate serious system problems that the application usually cannot recover from, so you should generally not try to catch them.

12. Share a real example where you used a finally block.

I commonly use finally when managing database connections or file streams. For instance, after opening a Connection to a database in a try block, I place the connection.close() call

inside finally. This ensures the connection closes properly even if the SQL query fails, preventing memory leaks.

GENERICS (INTERMEDIATE)

1. What is type erasure in generics?

Type erasure is the process where the Java compiler removes all generic type information (like <String> or <Integer>) after compiling the code. This means that at runtime, the JVM sees only raw types (e.g., just List instead of List<String>). This was done to ensure backward compatibility with older versions of Java that did not have generics.

2. Why can't we make arrays of generic types?

Arrays are **covariant** and check their element types at runtime, whereas generics are **invariant** and lose their type info at runtime (erasure). Because the runtime system wouldn't know the specific generic type of the array, it couldn't prevent you from storing the wrong type of object. To avoid these unsafe runtime errors, Java simply forbids generic arrays.

3. What are wildcards like ?, ? extends T, and ? super T?

The wildcard ? represents an **unknown type**. ? extends T is an **upper bound**, accepting T or any of its subclasses (useful for reading data safely). ? super T is a **lower bound**, accepting T or any of its superclasses (useful for adding data to a collection).

4. What are generics and why were they introduced?

Generics allow you to write classes and methods that can work with any data type by using a placeholder (like <T>). They were introduced to provide stronger **type safety** at compile time. This catches errors early and eliminates the need for manual type casting when retrieving objects from a collection.

5. What does generic type inference mean?

Type inference is the compiler's ability to look at the context of your code and automatically figure out the generic type arguments. A common example is the **diamond operator** (<>), where you can write new ArrayList<>() without repeating the type name, because the compiler infers it from the variable declaration.

6. What are bounded type parameters such as <T extends Number>?

Bounded type parameters restrict the kinds of types that can be used as arguments. For example, <T extends Number> tells the compiler that T can only be the class Number or one of its subclasses (like Integer or Double). This allows you to safely call methods defined in the Number class on the generic object.

7. How does List<Object> differ from List<?>?

List<Object> is a list that explicitly holds Objects; you can add anything to it, but you cannot pass a List<String> to a variable of this type. List<?> is a list of an **unknown type**; it is a generic supertype for all lists, but it is effectively **read-only** because the compiler doesn't know what specific type is safe to add to it.

JAVA 8 FEATURES (INTERMEDIATE)

1. What are some main features added in Java 8?

Java 8 introduced **Lambda Expressions** for functional programming and the **Streams API** for efficient bulk data processing. It also added the **Optional** class to handle null values better and a new, robust **Date and Time API**. Additionally, it allowed interfaces to have default and static methods for the first time.

2. What are functional interfaces?

A **Functional Interface** is an interface that contains exactly one abstract method. They act as the target type for lambda expressions and method references. Common examples built into Java include Runnable, Callable, and Comparator.

3. How do lambda expressions help developers?

Lambda expressions allow developers to write concise, anonymous methods directly in line with their code. This reduces "boilerplate" code (like anonymous inner classes) and enables a functional programming style. They make code easier to read and allow you to pass behavior as arguments to methods.

4. What are method references?

Method references are a shorthand syntax (using ::) for a lambda expression that executes just one existing method. Instead of writing s -> System.out.println(s), you can simply write System.out::println. They make the code even cleaner and more readable when you are just passing data to an existing function.

5. Why do lambda expressions need effectively final variables?

Lambdas capture the values of local variables, not the variables themselves. If the variable changed later, the lambda would be holding stale or inconsistent data, leading to synchronization issues. Therefore, Java enforces that local variables used in lambdas must not change (be **effectively final**) to ensure thread safety.

6. What is Optional and why do we use it?

Optional is a container object used to represent the presence or absence of a value. We use it to avoid NullPointerException by making it explicit that a return value might be missing. It encourages developers to handle the "empty" case explicitly using methods like .orElse() or .ifPresent().

7. What is the Streams API?

The **Streams API** is a powerful framework for processing sequences of elements (like collections) in a functional style. It allows you to perform complex operations like filtering, mapping, and reducing data using declarative code. It does not store data itself but computes it on demand as it flows through the pipeline.

8. How does map() differ from flatMap()?

map() transforms each element into exactly one new element (e.g., converting a list of Strings to a list of Integers). **flatMap()** transforms each element into a *stream* of elements and then flattens them into a single continuous stream. flatMap is essential when dealing with nested structures like a List<List<String>>.

9. How does filter() work in streams?

The **filter()** method takes a **Predicate** (a condition returning true/false) and keeps only the elements that match that condition. Elements that return false are discarded from the stream pipeline. It is used to select a specific subset of data, such as "all numbers greater than 10."

10. What are intermediate and terminal operations?

Intermediate operations (like filter or map) transform a stream into another stream and are lazy (they don't execute immediately). **Terminal operations** (like collect or forEach) trigger the actual processing and produce a final result or side effect, effectively closing the stream.

11. How do findFirst() and findAny() differ?

findFirst() deterministically returns the very first element in the stream (respecting insertion order). **findAny()** returns *any* element it finds; in parallel streams, this is faster because it returns whichever thread finishes first, without worrying about the original order.

12. What does forEach() do in streams?

forEach() is a terminal operation that iterates through every element in the stream and performs an action (like printing). Because it is a terminal operation, the stream is consumed and cannot be used again after forEach is called.

13. How do we sort data using streams?

We use the **sorted()** intermediate operation. Calling sorted() with no arguments sorts data in its natural order (like A-Z or 1-10). You can also pass a custom Comparator to sorted() to define specific sorting logic, such as sorting objects by a specific property.

14. What does Collectors.toList() do?

Collectors.toList() is a helper method used in the terminal .collect() operation. It gathers all the elements remaining in the stream and puts them into a new List implementation (usually an ArrayList). It is the standard way to get your processed data back out of a stream.

15. How can you convert a List to a Map using streams?

You use .collect(Collectors.toMap(keyMapper, valueMapper)). You must provide two functions: one to extract the **key** and one to extract the **value** from the stream elements. You must be careful to ensure keys are unique, otherwise, it will throw a duplicate key exception.

16. What is the difference between skip() and limit()?

limit(n) truncates the stream, allowing only the first n elements to pass through. **skip(n)** does the opposite; it discards the first n elements and passes the rest of the stream down the pipeline. They are often used together for pagination (e.g., "skip 10, take 10").

17. What are parallel streams and when should they be avoided?

Parallel streams split the data into chunks and process them simultaneously on multiple threads to speed up large tasks. They should be avoided for small datasets or simple tasks because the overhead of managing threads (context switching) can actually make them slower than sequential streams.

MULTITHREADING & CONCURRENCY (INTERMEDIATE)

1. What is a race condition?

A **race condition** is a bug that occurs when two or more threads access shared data at the same time and try to change it. The final result depends on the "race" or timing of which thread finishes first, making the outcome unpredictable and often incorrect. It typically happens when operations like "read-modify-write" are not atomic.

2. Why do we need thread synchronization?

We need **synchronization** to prevent race conditions and ensure data consistency. By restricting access so that only one thread can use a shared resource at a time, we ensure that one thread finishes its update completely before another thread sees or modifies that data.

3. What does the synchronized keyword do?

The synchronized keyword acts as a **lock** mechanism. When a method or block is marked as synchronized, a thread must acquire a specific lock before entering it. While that thread holds the lock, all other threads attempting to enter that code must wait until the lock is released.

4. What is the difference between a synchronized method and a synchronized block?

A **synchronized method** locks the entire method scope, using the object instance (this) or the class as the lock. A **synchronized block** allows you to lock only a specific, critical section of code within a method and lets you choose exactly which object to use as the lock. Blocks are generally preferred for better performance because they keep the "locked" time shorter.

5. What is a deadlock?

A **deadlock** is a situation where two or more threads are stuck waiting for each other forever. For example, Thread A holds Lock 1 and waits for Lock 2, while Thread B holds Lock 2 and waits for Lock 1. Neither can proceed, and the program effectively freezes.

6. What is thread starvation?

Thread starvation happens when a thread is perpetually denied access to CPU resources or locks because other threads are greedy. This usually occurs when high-priority threads constantly monopolize the CPU, leaving lower-priority threads waiting indefinitely without ever getting a chance to run.

7. What is a daemon thread?

A **daemon thread** is a background service thread (like the Garbage Collector) that supports the main application. Its lifecycle depends on user threads: if all standard "user" threads finish execution, the JVM terminates immediately, killing all daemon threads even if they are still running.

8. Why must wait() and notify() be inside synchronized blocks?

These methods rely on the object's **monitor** (lock) to coordinate communication between threads. You must be inside a synchronized block to guarantee that you currently hold that lock. If you call them without holding the lock, Java throws an IllegalMonitorStateException.

9. What is the purpose of ThreadLocal?

ThreadLocal allows you to create variables that can only be read and written by the same thread. Effectively, it gives each thread its own private copy of a variable. This is useful for storing context data (like a user ID or transaction ID) that should be accessible anywhere in that specific thread's execution path but isolated from other threads.

10. What happens if an exception occurs inside a synchronized block?

If an exception is thrown inside a synchronized block and is not caught, the JVM automatically **releases the lock** held by that thread. This is a safety feature that prevents the system from freezing up (deadlocking) due to a crashed thread holding onto a lock forever.

11. What is a volatile variable?

The volatile keyword guarantees **visibility** of changes to variables across threads. It ensures that if one thread changes a value, that change is immediately written to main memory, and other threads see the new value instantly. However, unlike synchronization, it does not guarantee atomicity (thread safety for complex operations).

12. How do we safely access shared data in a multithreaded program?

You can safely access shared data by using **synchronization** (locks) to enforce exclusive access. Alternatively, you can use the java.util.concurrent package, which provides thread-safe collections (like ConcurrentHashMap) and atomic variables (like AtomicInteger) that handle the safety logic for you.

13. What is the difference between wait(), sleep(), and yield()?

wait() releases the lock and pauses until notified. sleep() pauses the thread for a set time but **keeps the lock**, blocking others. yield() simply pauses the current thread momentarily to allow

other threads of the same priority to execute, but it does not release locks or guarantee when it will run again.

DESIGN PATTERNS (INTERMEDIATE)

1. What is the Factory Pattern?

The **Factory Pattern** is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created. Instead of calling the constructor directly (using new), you call a factory method that decides which object to return. This decouples the client code from the specific classes it needs to instantiate.

2. Why do we need design patterns in software development?

Design patterns are tried-and-tested solutions to common problems encountered in software design. They provide a shared vocabulary for developers (e.g., saying "use a Singleton" instantly communicates the design intent). Using them makes code more reusable, maintainable, and easier to understand for new team members.

3. What is the Singleton Pattern?

The **Singleton Pattern** ensures that a class has exactly one instance and provides a global point of access to it. It is useful when exactly one object is needed to coordinate actions across the system, such as a configuration manager or a logger.

4. Can Singleton be thread-safe without extra handling?

No, the standard "lazy initialization" Singleton (where you create the instance only when asked) is **not** thread-safe by default. If two threads try to access the creation method simultaneously for the first time, they might both create an instance, resulting in two distinct objects and violating the pattern.

5. How can we make a Singleton thread-safe?

You can make a Singleton thread-safe by marking the creation method as synchronized, ensuring only one thread enters at a time. A more efficient approach is **Double-Checked**

Locking, which only synchronizes during the first creation. Alternatively, using an **Enum Singleton** is the easiest and safest way, as Java handles the thread safety internally.

6. Why is Singleton used in database connection pools?

Creating a new database connection for every query is incredibly slow and resource-heavy. A **Singleton** connection pool manager ensures there is only one centralized place to create, manage, and recycle a limited number of open connections. This prevents the system from being overwhelmed by too many simultaneous connections.

7. How does the Builder pattern help create objects?

The **Builder Pattern** helps construct complex objects step-by-step, which is especially useful when a class has many parameters (some mandatory, some optional). Instead of a confusing constructor with 10 arguments (new Car(true, false, 4, "Red"...)), you use readable methods like .setColor("Red").setDoors(4).build(). It makes the code significantly more readable and less error-prone.

SERIALIZATION (INTERMEDIATE)

1. What is serialization?

Serialization is the process of converting the state of an object into a byte stream (a sequence of bytes). This allows the object to be saved to a file, stored in a database, or sent over a network. The reverse process, where the byte stream is converted back into a live Java object, is called **deserialization**.

2. What does serialVersionUID represent?

serialVersionUID is a unique version identifier for a Serializable class. It acts like a checksum to ensure that the class definition used to serialize the object matches the class definition used to deserialize it. If the versions do not match (e.g., you added a method to the class after saving the object), Java throws an InvalidClassException.

3. What does the transient keyword do?

The **transient** keyword tells the JVM to **skip** a specific variable during the serialization process. If a field is marked as transient, its value is not saved to the byte stream. This is commonly used for sensitive data (like passwords) or temporary data that doesn't need to be persisted.

4. How can serialization break a Singleton pattern?

Serialization breaks the **Singleton pattern** because deserializing an object creates a **new instance** of the class by default, bypassing the private constructor. If you serialize your Singleton and then deserialize it later, you will end up with two different instances (the original and the deserialized copy), violating the rule that only one instance should exist.

5. How can we stop a new instance from being created during deserialization?

To fix the Singleton issue, you must implement the **readResolve()** method in your class. This special method is called implicitly by the JVM immediately after an object is deserialized. By returning the existing Singleton instance from this method, you force Java to discard the newly created duplicate.

6. How is Externalizable different from Serializable?

Serializable is a marker interface that hands over the entire process to the JVM, which is easier but slower. **Externalizable** extends Serializable but requires the programmer to manually define the logic using writeExternal() and readExternal() methods. Externalizable allows for complete control and is significantly faster because you serialize only what is strictly necessary.

JVM, MEMORY & CLASSLOADER (INTERMEDIATE)

1. What does the JIT compiler do?

The **JIT (Just-In-Time) Compiler** is a performance component of the JVM that speeds up execution. Instead of interpreting bytecode line-by-line every time, the JIT compiles frequently used code ("hot spots") directly into native machine code. This allows the program to run at speeds close to native applications like C++.

2. What are the different types of ClassLoaders?

There are three main built-in class loaders in Java. The **Bootstrap ClassLoader** loads core Java libraries (like java.lang.*). The **Extension (or Platform) ClassLoader** loads external extensions, and the **Application (System) ClassLoader** loads the classes found in your project's classpath.

3. What is a ClassLoader?

A **ClassLoader** is a subsystem of the JVM responsible for finding and loading class files (.class) into memory at runtime. It follows a "lazy loading" approach, meaning it only loads a class when the application explicitly needs it. Without it, the JVM wouldn't know how to read your compiled code.

4. What is a stack frame?

A **stack frame** is a block of memory created within the Stack whenever a method is called. It contains the specific state for that method invocation, including local variables, method arguments, and the return address. Once the method finishes execution, the frame is immediately destroyed.

5. How is heap memory different from stack memory?

Stack memory is small, fast, and thread-specific; it manages method execution and local variables. **Heap memory** is large and shared across all threads; it stores all the actual **objects** created by the application. While the Stack cleans itself up automatically, the Heap relies on the Garbage Collector.

6. What is Metaspace and how is it different from PermGen?

Metaspace was introduced in Java 8 to replace **PermGen** for storing class metadata (descriptions of your classes). Unlike PermGen, which was part of the fixed-size Heap, Metaspace uses **native memory** provided by the OS. This allows it to grow automatically, significantly reducing OutOfMemoryError crashes related to class loading.

7. What causes OutOfMemoryError related to heap?

This error (Java heap space) happens when the application creates more objects than the Heap can hold, and the Garbage Collector cannot free up enough space. It is usually caused

by a **memory leak** (keeping references to unused objects) or trying to process too much data (like a massive file) all at once.

8. What causes OutOfMemoryError related to Metaspace?

This error (Metaspace) occurs when the JVM runs out of native memory to store class metadata. It typically happens in applications that generate a huge number of dynamic classes at runtime (often via reflection or proxies in frameworks like Spring). It can also occur if you have a **ClassLoader leak** where old classes aren't unloaded.

MISCELLANEOUS (INTERMEDIATE)

Here are the answers for the **Inner Classes & Design Principles** section of your tool kit.

1. What are inner classes?

An **inner class** is a class defined inside another class. It is used to logically group classes that are only used in one place, keeping the code organized. A key feature is that an inner class can access **private** members of the outer class that contains it.

2. What is a static nested class?

A **static nested class** is a static class defined inside another class. Unlike a regular inner class, it does **not** require an instance of the outer class to be created first. It behaves exactly like a normal top-level class but is nested inside another class purely for packaging and grouping convenience.

3. What is an anonymous inner class?

An **anonymous inner class** is a class without a name that is defined and instantiated in a single step. It is typically used for quick, one-time implementations, such as creating an event listener for a button click. It saves you from writing a separate class file for a tiny piece of logic that is only used once.

4. Why should we override toString() in our classes?

You should override toString() to provide a meaningful, human-readable description of your object (e.g., "User: John, ID: 50"). By default, Java prints a cryptic internal memory address code which is useless for humans. Overriding it makes **debugging** and logging significantly easier because you can instantly see the object's state.

5. What is a marker interface and why was it used earlier?

A **marker interface** is an empty interface that has no methods or constants (e.g., Serializable, Cloneable). It is used solely to signal to the JVM or compiler that a class should be treated in a special way. In modern Java, **Annotations** (like @Entity) have largely replaced marker interfaces because they are more flexible.

6. What does “composition over inheritance” mean?

This is a design principle that suggests you should build classes by combining simple objects ("has-a" relationship) rather than extending complex parent classes ("is-a" relationship). Inheritance creates tight coupling, meaning changes in a parent class can easily break child classes. Composition is more flexible, easier to test, and avoids this fragility.

7. What is defensive copying?

Defensive copying is the practice of creating a copy of a mutable object before returning it to the caller or storing it. This ensures that if external code modifies the object they received, your class's internal data remains safe and unchanged. It is a critical technique for maintaining **immutability**.

8. What is the difference between shallow copy and deep copy?

A **shallow copy** creates a new object but copies only the *references* to the child objects, meaning both the original and the copy share the same internal data. A **deep copy** creates a new object and recursively creates copies of all the child objects as well. Deep copy ensures the two objects are completely independent of each other.

9. What are static imports?

Static imports allow you to use static members (methods and fields) from other classes without qualifying them with the class name. For example, instead of typing Math.sqrt(), you can use import static java.lang.Math.*; and then simply type sqrt(). This keeps code cleaner when using many utility functions or constants.

10. Why should mutable objects not be used as keys in a Map?

If a key object changes its internal state, its **hash code** typically changes as well. When you try to find that key later, the Map will look in the wrong "bucket" based on the new hash code and will fail to find the entry. This effectively causes data loss, as the value exists but becomes unretrievable.

11. What logging levels are commonly used in Java?

The standard logging levels, ordered by severity, are **TRACE** (deepest detail), **DEBUG** (developer info), **INFO** (general flow), **WARN** (potential issues), and **ERROR** (failures). Using these levels allows you to filter logs; for example, in production, you might only turn on ERROR and INFO to save disk space, while using DEBUG locally.